

Simulação e Teste de Software (CC8550)

Introdução à Disciplina

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Por Que Testar Software?

Uma disciplina crítica, não coadjuvante

- Software controla aviões, marca-passos, usinas, carros, bancos e eleições
- Cada linha de código carrega uma promessa: *isto vai funcionar como deveria*
- Quando essa promessa é quebrada, as consequências vão de um aborrecimento a uma tragédia
- Testar é como verificamos, de forma sistemática, se a promessa está sendo cumprida

Falhas que Marcaram a História (I)

Therac-25 (1985–1987)

Equipamento de radioterapia com falha de *race condition* no software de controle; pacientes receberam doses de radiação até 100 vezes acima do previsto. Ao menos seis acidentes documentados, com mortes.

Ariane 5, Voo 501 (1996)

Código reaproveitado do Ariane 4 causou um *overflow* ao converter um número de 64 bits para 16 bits. O foguete se autodestruíu 37 segundos após o lançamento – prejuízo estimado em US\$ 370 milhões.

Falhas que Marcaram a História (II)

Mars Climate Orbiter (1999)

Uma equipe usou unidades imperiais (libra-força) e outras unidades métricas (newton) no software de navegação. A sonda se perdeu ao entrar na atmosfera de Marte – missão de US\$ 327 milhões perdida.

Knight Capital (2012)

Uma implantação de código deixou uma rotina antiga ativa por engano em um dos servidores. Em 45 minutos, ordens erráticas de compra e venda causaram um prejuízo de US\$ 440 milhões e quase quebraram a empresa.

Falhas que Marcaram a História (III)

Heartbleed (2014)

Falha na biblioteca OpenSSL permitia ler trechos da memória de servidores – incluindo senhas e chaves privadas. Afetou cerca de 500 mil servidores em todo o mundo.

Boeing 737 MAX – MCAS (2018–2019)

Sistema de software (MCAS) confiava em um único sensor de ângulo de ataque. Dois acidentes – Lion Air e Ethiopian Airlines – mataram 346 pessoas e levaram ao maior recall da aviação comercial.

Outros exemplos conhecidos

- **Bug do Milênio (Y2K):** anos representados com 2 dígitos ameaçavam confundir 2000 com 1900 – mais de US\$ 300 bilhões investidos no mundo todo para prevenir falhas
- **Aceleração não intencional da Toyota (2005–2010):** perícia encontrou falhas no software de controle do acelerador associadas a acidentes fatais
- Estes são só os casos que *viraram notícia* – a maioria dos bugs nunca é manchete, mas custa tempo, dinheiro e confiança todos os dias

O Custo de um Defeito Cresce com o Tempo

Quanto mais tarde encontrado, mais caro fica corrigir

Fase em que o defeito é encontrado	Custo relativo
Requisitos	1x
Projeto (design)	5x
Implementação	10x
Testes	15x – 40x
Produção (pós-lançamento)	30x – 100x+

Valores ilustrativos, baseados em Boehm (1981). A proporção exata varia por projeto, mas a tendência é consistente: **corrigir cedo é sempre mais barato que corrigir tarde.**

Testar é Prevenir, Não Remediar

O que está realmente em jogo

- Confiança: usuários, clientes e a sociedade dependem do software funcionar
- Segurança: em sistemas críticos, um bug pode custar vidas
- Dinheiro: encontrar um defeito antes do lançamento é sempre mais barato
- Reputação: uma falha pública pode levar anos para ser esquecida

Nesta disciplina

Você vai aprender a pensar como testador: a antecipar o que pode dar errado antes que isso aconteça.

IA e o Futuro do Teste de Software

Um cenário em rápida mudança

- Assistentes de IA (GitHub Copilot, Cursor, Claude, etc.) já geram testes automaticamente a partir do código
- Ferramentas de teste baseadas em IA sugerem casos, dados de entrada e até corrigem testes quebrados
- Produtividade aumenta – gerar um teste unitário simples pode levar segundos
- Pergunta natural: *a IA vai substituir o testador humano?*

O Problema do Oráculo

O que é um “oráculo” de teste?

É o critério que diz se um resultado está certo ou errado. Sem ele, não existe teste – só execução.

Por que isso é difícil para a IA

- Requisitos de negócio muitas vezes são ambíguos, implícitos ou vivem só na cabeça do cliente
- A IA aprende padrões de código, não a intenção de negócio por trás dele
- Sem um oráculo confiável, a IA pode gerar testes que *passam* sem verificar o que realmente importa

Riscos comuns de testes gerados por IA

- Testes sintaticamente corretos, mas semanticamente vazios (só confirmam o que o código já faz, certo ou errado)
- “Alucinação”: a IA assume um comportamento que o sistema não tem
- Casos redundantes que inflam a métrica de cobertura sem aumentar a capacidade de achar bugs
- Resultado: uma falsa sensação de segurança – *coverage theater*

O que a IA tende a não enxergar

- Casos extremos que exigem **conhecimento de domínio**: regras de negócio, contexto cultural, regulação
- Cenários adversariais: o que um usuário mal-intencionado faria?
- “Unknown unknowns”: bugs que ninguém pensou em perguntar, por estarem fora do padrão aprendido
- Teste exploratório: intuição construída pela experiência, não por estatística de treinamento

Quem responde quando o software falha?

Em sistemas críticos – saúde, aviação, finanças – alguém precisa assinar embaixo. Esse alguém é uma pessoa, não um modelo de IA.

Na prática

- Normas de certificação (ex.: DO-178C na aviação, IEC 62304 em software médico) exigem rastreabilidade e julgamento humano
- Priorizar riscos, decidir o que testar primeiro e quando “é suficiente” são decisões estratégicas, não apenas técnicas

O modelo que funciona: humano no controle

- IA acelera: gera esqueletos de testes, sugere casos, automatiza repetição
- Humano decide: qual é o oráculo correto, quais riscos priorizar, o que a IA deixou passar
- Testadores que dominam as técnicas certas usam IA como multiplicador, não como atalho

É exatamente isso que você vai construir aqui

As técnicas desta disciplina são o que te permite avaliar, direcionar e confiar – ou não – em qualquer teste, gerado por você ou por uma IA.

Roteiro da Disciplina

- 1 Fundamentos e Taxonomia de Testes de Software
- 2 Ciclo de Vida de Teste de Software (STLC)
- 3 Planejamento Mestre de Testes e Análise de Risco
- 4 Introdução à Automação de Testes com pytest
- 5 Teste de Unidade Avançado (FIRST, padrão AAA)
- 6 Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite
- 7 Técnicas Caixa-Preta: Teste Combinatorial e Property-Based
- 8 Técnicas Caixa-Branca: Critérios de Cobertura Estrutural
- 9 Testes de Mutação e Avaliação da Eficácia de Suítes de Teste
- 10 Test-Driven Development (TDD)
- 11 Teste de Integração, Test Doubles e Níveis de Teste
- 12 Teste Orientado a Objetos e de Componentes

Ementa e Objetivos

O que a disciplina cobre

- Validação e verificação de software: técnicas e critérios
- Testes funcionais, baseados em modelo, estruturais e de mutação
- Testes orientados a objetos, de componentes e de aplicações web
- Laboratório acompanha as aulas teóricas, com prática direta de cada técnica

Ao final da disciplina, você deve ser capaz de

- Planejar um ciclo de testes para um sistema real, com análise de risco
- Aplicar as principais técnicas de teste caixa-preta e caixa-branca
- Automatizar testes com pytest e avaliar a eficácia de uma suíte de testes
- Praticar Test-Driven Development e testar sistemas orientados a objetos

Teoria e laboratório

- Encontros de teoria: aulas expositivas, com exemplos em sala
- Encontros de laboratório: aplicação prática do que foi visto na teoria mais recente
- Material de apoio complementar disponível no Moodle da disciplina

Avaliação

Como a nota é composta

- Atividades práticas em aula
- Prova escrita (P1 e P2)
- Feedback individual para cada atividade

Para ser aprovado

- Frequência mínima de 75% nas aulas
- Nota final maior ou igual a 5,0

O que se espera de você

- Participação ativa nas aulas teóricas e práticas
- Resolução dos exercícios propostos

Livros-texto

- SINGH, Yogesh. *Software Testing*. Cambridge University Press, 2011.
- DELAMARO, M. E.; MALDONADO, J. C.; JINO, M. (Org.). *Introdução ao Teste de Software*. 2. ed. Elsevier, 2016.

Leitura adicional

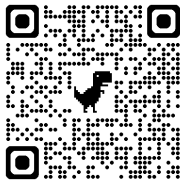
- CRAIG, R. D.; JASKIEL, S. P. *Systematic Software Testing*. Artech House, 2002.
- HASS, A. M. J. *Guide to Advanced Software Testing*. Artech House, 2008.

Pesquisa de Egressos

Ajude o curso com sua participação

Formulário de Egressos

- Participação voluntária e totalmente anônima
- Dado importante para o curso – vale a pena todo mundo responder
- Aponte a câmera do celular para o QR Code abaixo



`form-egresso.vercel.app`

Simulação e Teste de Software (CC8550)

Introdução à Disciplina

Prof. Luciano Rossi

Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026